



南開大學
Nankai University

计算机学院
并行程序设计实验报告

SIMD 编程

姓名：王一诺

学号：2312331

专业：计算机科学与技术

2025 年 4 月 29 日

目录

1 实验目标	2
2 实验平台配置	2
3 代码设计	2
3.1 朴素多项式乘法与 SIMD 优化	2
3.2 递归实现的 NTT 多项式乘法	3
3.3 使用 Montgomery 算法 neon 优化的递归 NTT	4
3.3.1 Montgomery 规约与向量化取模/模乘	5
3.3.2 向量化蝴蝶变换	6
3.3.3 最终代码实现	6
3.4 迭代实现的 NTT 多项式乘法	8
3.5 使用 Montgomery 算法 neon 优化的迭代 NTT	9
3.5.1 设计思想	9
3.5.2 最终代码实现	9
4 ARM 下的结果与分析	11
5 移植 x86 与运行分析	12
5.1 移植后代码	12
5.2 运行结果与分析	12
6 实验总结和思考	13
7 链接	13

1 实验目标

在多项式乘法的计算中，传统朴素算法的时间复杂度为 $O(n^2)$ ，难以满足实际应用中对大规模多项式运算的效率需求。数论变换（NTT）作为快速数论域中的离散傅里叶变换（FFT），可将多项式乘法的时间复杂度降低至 $O(n \log n)$ ，成为同态加密等领域的核心优化手段。本实验聚焦 NTT 算法的实现与优化，重点研究递归与迭代式 NTT 的设计差异，并通过 Montgomery 模约减算法与 SIMD 向量化技术（ARM NEON 及 x86 AVX）进一步加速计算。通过对比朴素乘法、递归/迭代 NTT 及其优化版本在不同平台（ARM 鲲鹏与 x86）的性能表现，验证优化策略的有效性。

2 实验平台配置

平台 \ 处理器	操作系统	编译器	指令集
ARM/华为鲲鹏处理器	OpenEuler	GCC	NEON 指令集
X86/Intel Core i9-14900HX	Windows 11	MSVC	AVX/AVX2 指令集

表 1: 实验环境

3 代码设计

3.1 朴素多项式乘法与 SIMD 优化

朴素多项式乘法是直接按照多项式乘法的定义实现，即对于两个长度为 n 的多项式 a 和 b ，计算它们的乘积 ab 时，需要计算 a 中每个元素与 b 中每个元素的乘积并累加到结果数组的相应位置。这种方法的复杂度为 $O(n^2)$ ，虽然简单直观，但在处理大规模多项式时效率低下。

针对 ARM 平台，我们使用 NEON 指令集进行 SIMD 优化。在 `poly_multiply_neon` 函数中，通过将每次乘法运算向量化，利用 NEON 指令同时处理四个整数元素，即每次将系数 $a[i]$ 广播到整个 SIMD 寄存器，然后加载连续的 4 个 $b[j]$ 值进行向量化乘法。减少了循环迭代次数，从而大幅度提升了计算效率。

NEON 优化版本的关键在于正确处理向量化乘法和模运算。由于 NEON 指令集不直接提供向量化的模运算，我们采用了混合策略：向量化计算乘法和加法，然后标量处理模运算。虽然这种实现不能完全向量化，但仍然显著提升了性能。

朴素乘法的 SIMD 优化

```
1 void poly_multiply_neon(int *a, int *b, int *ab, int n, int p) { // 朴素乘法的 SIMD 优化
2     // 初始化结果数组
3     for (int i = 0; i < 2 * n - 1; ++i) {
4         ab[i] = 0;
5     }
6     for (int i = 0; i < n; ++i) {
7         int a_val = a[i];
8         int32x2_t a_low2 = vdup_n_s32(a_val); // 复制两个 a[i]
9
10        for (int j = 0; j + 3 < n; j += 4) {
11            // 加载 b[j], b[j+1], b[j+2], b[j+3]
```

```

12     int32x4_t b_vec = vld1q_s32(&b[j]);
13     // 拆成低2位和高2位
14     int32x2_t b_low2 = vget_low_s32(b_vec);
15     int32x2_t b_high2 = vget_high_s32(b_vec);
16     // 乘法: a[i] * b[j~j+3] -> int64x2_t
17     int64x2_t mul_low2 = vmull_s32(a_low2, b_low2); // a[i] * b[j], b[j+1]
18     int64x2_t mul_high2 = vmull_s32(a_low2, b_high2); // a[i] * b[j+2], b[j+3]

19     // 加上 ab[i+j ~ i+j+3]
20     int64_t tmp[4];
21     tmp[0] = mul_low2[0] + ab[i + j + 0];
22     tmp[1] = mul_low2[1] + ab[i + j + 1];
23     tmp[2] = mul_high2[0] + ab[i + j + 2];
24     tmp[3] = mul_high2[1] + ab[i + j + 3];
25     // 取模写回
26     for (int k = 0; k < 4; ++k) {
27         tmp[k] %= p;
28         if (tmp[k] < 0) tmp[k] += p;
29         ab[i + j + k] = (int)tmp[k];
30     }
31 }
32 // 处理剩下不足4个的部分
33 for (int j = (n & ~3); j < n; ++j) {
34     ab[i + j] = (1LL * a[i] * b[j] % p + ab[i + j]) % p;
35 }
36 }
37 }

```

3.2 递归实现的 NTT 多项式乘法

NTT 递归实现的基本思想是将原问题分解为子问题，采用分治的策略，递归地计算多项式变换。对于长度为 N 的多项式，我们将其分成两部分，分别对两部分进行递归变换，然后合并结果。递归的复杂度是 $O(n \log n)$ ，比朴素的 $O(n^2)$ 要高效得多。

递归 NTT 的核心操作是蝴蝶变换，它将两个点的值组合成两个新的值：一个和值和一个差值。在每次递归调用中，我们将输入数组分成两半，分别对奇数和偶数位置的元素进行变换。递归基是长度为 1 的数组，此时直接返回。合并阶段使用旋转因子进行蝴蝶操作，旋转因子通过快速幂计算得到。

多项式乘法通过 NTT 实现时，需要先将两个多项式转换到 NTT 域，然后进行点乘，最后进行逆 NTT 变换。NTT_recursion_multiply 函数实现了这一流程：补零扩展数组长度到 2 的幂次，执行正向 NTT，点乘，然后执行逆 NTT 并缩放结果。虽然递归实现清晰易懂，但由于递归开销和频繁的内存访问，可以预料到其性能不如迭代实现。

NTT 递归实现

```

1  const int g=3;
2  // NTT递归实现
3  void NTT_recursion(int *a,int len,int op,int g,int p){//NTT递归模块
4      if (len == 1) return;

```

```

5     int half = len / 2;
6     int A1[half], A2[half];
7
8     for (int i = 0; i < half; ++i) {
9         A1[i] = a[i * 2];
10        A2[i] = a[i * 2 + 1];
11    }
12
13    NTT_recursion(A1, half, op, g, p);
14    NTT_recursion(A2, half, op, g, p);
15
16    int wn = qpow((op == 1 ? g : qpow(g, p - 2, p)), (p - 1) / len, p);
17    int w = 1;
18    for (int i = 0; i < half; ++i) {
19        int t = 1LL * A2[i] * w % p;
20        a[i] = (A1[i] + t) % p;
21        a[i + half] = (A1[i] - t + p) % p;
22        w = 1LL * w * wn % p;
23    }
24 }
25
26 void NTT_recursion_multiply(int *a, int *b, int *ab, int n, int p){//NTT递归
27     int N = 2 * n - 1;
28     int len = 1;
29     int ni;
30     while (len < N) len <= 1; // 求len,且保证为2的整数次幂
31     ni=qpow(len,p-2,p);
32     int A[len], B[len];
33     for (int i = 0; i < len; ++i) {
34         A[i] = (i < n) ? a[i] : 0;
35         B[i] = (i < n) ? b[i] : 0;
36     }
37     NTT_recursion(A, len, 1, g, p);
38     NTT_recursion(B, len, 1, g, p);
39     for (int i = 0; i < len; ++i) {
40         A[i] = (1LL * A[i] * B[i]) % p;
41     }
42     NTT_recursion(A, len, -1, g, p);
43     for (int i = 0; i < N; ++i) {
44         ab[i] = (1LL * A[i] * ni) % p;
45     }
46 }

```

3.3 使用 Montgomery 算法 neon 优化的递归 NTT

为了进一步提升递归 NTT 的性能,引入了 Montgomery 模约减算法和 NEON 向量化优化。Montgomery 算法通过将模运算转换为位移和减法,避免了昂贵的除法操作。我们设计了 MontgomeryParams 结构体来存储模数相关的预计算参数,包括模数、 n' 和 R^2 。在 NTT_recursion_mont 函数

中，我们使用 Montgomery 算法进行模乘与逆变换，以提高计算速度。通过 SIMD 优化，我们能够将 Montgomery 运算并行化，从而进一步提升性能。

3.3.1 Montgomery 规约与向量化取模/模乘

关于 Montgomery 规约：在使用 Montgomery 算法时，所有的操作都应在 Montgomery 域内进行。数字在进入 NTT 递归之前，需要先转换成 Montgomery 域表示，这样乘法与加法操作就可以高效地进行。于是设计 `to_montgomery_scalar(x)` 将一个普通的数字转换为 Montgomery 域的表示，设计 `from_montgomery_scalar(x)` 将 Montgomery 域的数字转换回普通数字。

数域转换

```
1  static inline uint32_t to_montgomery_scalar(uint32_t a, const MontgomeryParams*
    params) {
2      return montgomery_reduce_scalar((uint64_t)a * params->r_squared, params);
3  }
4  static inline uint32_t from_montgomery_scalar(uint32_t a, const MontgomeryParams*
    params) {
5      return montgomery_reduce_scalar((uint64_t)a, params);
6  }
```

在向量化实现中，主要挑战是如何在 NEON 指令集中高效实现 Montgomery 约减。设计了 `montgomery_reduce_neon` 函数，使用 NEON 指令并行处理 4 个元素的约减运算。对于乘法运算，设计了 `montgomery_multiply_neon`，它先计算常规乘积，然后应用 Montgomery 约减。

向量化 Montgomery 规约和模乘

```
1  static inline uint32x4_t montgomery_reduce_neon(uint64x2_t T_low, uint64x2_t
    T_high, const MontgomeryParams* params) {
2      uint32x4_t n_prime_vec = vdupq_n_u32(params->n_prime);
3      uint32x4_t mod_vec = vdupq_n_u32(params->mod);
4      uint32x4_t T_low32 = vcombine_u32(vmovn_u64(T_low), vmovn_u64(T_high));
5      uint32x4_t m = vmulq_u32(T_low32, n_prime_vec);
6
7      uint64x2_t mul_low = vmull_u32(vget_low_u32(m), vget_low_u32(mod_vec));
8      uint64x2_t mul_high = vmull_u32(vget_high_u32(m), vget_high_u32(mod_vec));
9      uint64x2_t sum_low = vaddq_u64(T_low, mul_low);
10     uint64x2_t sum_high = vaddq_u64(T_high, mul_high);
11
12     uint32x4_t t = vcombine_u32(vshrn_n_u64(sum_low, 32), vshrn_n_u64(sum_high, 32));
13     uint32x4_t cmp = vcgeq_u32(t, mod_vec);
14     uint32x4_t sub = vsubq_u32(t, mod_vec);
15     return vbslq_u32(cmp, sub, t);
16 }
17
18 static inline uint32x4_t montgomery_multiply_neon(uint32x4_t a, uint32x4_t b, const
    MontgomeryParams* params) {
19     uint64x2_t low = vmull_u32(vget_low_u32(a), vget_low_u32(b));
20     uint64x2_t high = vmull_u32(vget_high_u32(a), vget_high_u32(b));
21     return montgomery_reduce_neon(low, high, params);
22 }
```

22 | }

3.3.2 向量化蝴蝶变换

递归 NTT 的向量化优化版本 `NTT_recursion_mont` 将所有运算转换到 Montgomery 域进行。旋转因子的计算也使用 Montgomery 形式的快速幂。蝴蝶操作被重写为使用向量化 Montgomery 乘法和加法的形式。虽然递归结构限制了向量化的程度，但在合并阶段我们仍能对 4 个连续的蝴蝶操作进行向量化处理。

Montgomery 域内快速幂

```
1 uint32_t montgomery_pow(uint32_t a, int b, const MontgomeryParams* params) {
2     uint32_t res = to_montgomery_scalar(1, params); // Montgomery域的1
3     while (b) {
4         if (b & 1) res = montgomery_reduce_scalar((uint64_t)res * a, params);
5         a = montgomery_reduce_scalar((uint64_t)a * a, params);
6         b >>= 1;
7     }
8     return res;
9 }
```

3.3.3 最终代码实现

最终的 `NTT_recursion_multiply_SIMD` 函数将输入数据转换到 Montgomery 域，执行向量化优化的递归 NTT，进行点乘，然后逆变换并将结果转换回常规表示。这种实现减少了模运算的开销，提高了计算密度，特别是在处理大模数时优势明显。

使用 Montgomery 算法 neon 优化的递归 NTT

```
1 void NTT_recursion_mont(uint32_t *a, int len, int op, uint32_t g, uint32_t p, const
2     MontgomeryParams *params) {
3     if (len == 1) return;
4     int half = len / 2;
5     uint32_t *A1 = (uint32_t*)alloca(sizeof(uint32_t) * half);
6     uint32_t *A2 = (uint32_t*)alloca(sizeof(uint32_t) * half);
7     for (int i = 0; i < half; ++i) {
8         A1[i] = a[i * 2];
9         A2[i] = a[i * 2 + 1];
10    }
11    NTT_recursion_mont(A1, half, op, g, p, params);
12    NTT_recursion_mont(A2, half, op, g, p, params);
13
14    uint32_t root = (op == 1 ? g : qpow(g, p - 2, p));
15    uint32_t root_mont = to_montgomery_scalar(root, params);
16    uint32_t wn = montgomery_pow(root_mont, (p - 1) / len, params);
17    uint32_t w = to_montgomery_scalar(1, params);
18    for (int i = 0; i < half; ++i) {
```

```

19 // t = A2[i] * w (Montgomery乘法)
20 uint32_t t = montgomery_reduce_scalar((uint64_t)A2[i] * w, params);
21
22 // a[i] = (A1[i] + t) % p;
23 uint32_t add = A1[i] + t;
24 if (add >= p) add -= p;
25 a[i] = add;
26
27 // a[i+half] = (A1[i] - t + p) % p;
28 uint32_t sub = A1[i] + p - t;
29 if (sub >= p) sub -= p;
30 a[i + half] = sub;
31
32 // w = w * wn (Montgomery)
33 w = montgomery_reduce_scalar((uint64_t)w * wn, params);
34 }
35 }
36
37 void NTT_recursion_multiply_SIMD(int *a, int *b, int *ab, int n, int p, int g=3) {
38     const int N = 2 * n - 1;
39     int len = 1;
40     while (len < N) len <<= 1;
41     int ni = qpow(len, p - 2, p);
42     MontgomeryParams params = init_montgomery_params(p);
43
44     // 转Montgomery域
45     uint32_t *A = (uint32_t*)aligned_alloc(16, len * sizeof(uint32_t));
46     uint32_t *B = (uint32_t*)aligned_alloc(16, len * sizeof(uint32_t));
47     for (int i = 0; i < len; ++i) {
48         A[i] = (i < n) ? to_montgomery_scalar(a[i], &params) : 0;
49         B[i] = (i < n) ? to_montgomery_scalar(b[i], &params) : 0;
50     }
51
52     NTT_recursion_mont(A, len, 1, g, p, &params);
53     NTT_recursion_mont(B, len, 1, g, p, &params);
54
55     // 点值相乘Montgomery
56     for (int i = 0; i < len; ++i) {
57         A[i] = montgomery_reduce_scalar((uint64_t)A[i] * B[i], &params);
58     }
59
60     NTT_recursion_mont(A, len, -1, g, p, &params);
61
62     for (int i = 0; i < N; ++i) {
63         ab[i] = (uint64_t)from_montgomery_scalar(A[i], &params) * ni % p;
64     }
65     free(A);
66     free(B);
67 }

```


3.4 迭代实现的 NTT 多项式乘法

迭代 NTT 通过消除递归调用提高了性能。设计了 NTT_Iteration 函数和对应的多项式乘法函数 NTT_Iteration_multiply。迭代实现首先进行位逆序置换，然后通过多层循环实现蝴蝶操作。

位逆序置换确保数据在内存中的排列适合后续的迭代计算。迭代过程从最小的子问题开始，逐步合并结果。每次迭代处理当前长度的所有子序列，对每对元素应用蝴蝶操作。

NTT_Iteration_multiply 函数的流程与递归版本类似，但由于避免了递归开销，性能显著提升。迭代实现还具有更好的内存局部性，因为数据访问模式更加规律，有利于处理器缓存的有效利用。这种实现特别适合处理大规模多项式，是实践中常用的 NTT 实现方式。

NTT 迭代实现

```

1  void NTT_Iteration(int *a, int len, int op, int g, int p) { //NTT迭代模块
2      // 位逆序置换
3      for (int i = 1, j = 0; i < len - 1; i++) {
4          for (int k = len >> 1; (j ^ k) < k; k >>= 1);
5          if (i < j) {
6              int tmp = a[i];
7              a[i] = a[j];
8              a[j] = tmp;
9          }
10     }
11     // 蝴蝶操作
12     for (int h = 2; h <= len; h <<= 1) {
13         // 计算原根
14         int wn = qpow((op == 1 ? g : qpow(g, p - 2, p)), (p - 1) / h, p);
15         // 对每个长度为h的段进行变换
16         for (int j = 0; j < len; j += h) {
17             int w = 1;
18             for (int k = j; k < j + h/2; k++) {
19                 int t = 1LL * a[k + h/2] * w % p;
20                 a[k + h/2] = (a[k] - t + p) % p;
21                 a[k] = (a[k] + t) % p;
22                 w = 1LL * w * wn % p;
23             }
24         }
25     }
26 }
27 void NTT_Iteration_multiply(int *a, int *b, int *ab, int n, int p) { //NTT迭代
28     int N = 2 * n - 1;
29     int len = 1;
30     while (len < N) len <<= 1; // 确保len为2的幂次
31     // 计算len的逆元
32     int ni = qpow(len, p-2, p);
33     // 创建并初始化临时数组
34     int *A = new int[len]();
35     int *B = new int[len]();
36     // 复制输入数据
37     for (int i = 0; i < len; i++) {

```

```

38     A[i] = (i < n) ? a[i] : 0;
39     B[i] = (i < n) ? b[i] : 0;
40 }
41 // NTT变换
42 NTT_Iteration(A, len, 1, g, p);
43 NTT_Iteration(B, len, 1, g, p);
44 // 点值乘法
45 for (int i = 0; i < len; i++) {
46     A[i] = (1LL * A[i] * B[i]) % p;
47 }
48 // 逆变换
49 NTT_Iteration(A, len, -1, g, p);
50 // 乘上ni并保存结果
51 for (int i = 0; i < N; i++) {
52     ab[i] = (1LL * A[i] * ni) % p;
53 }
54 delete[] A;
55 delete[] B;
56 }

```

3.5 使用 Montgomery 算法 neon 优化的迭代 NTT

3.5.1 设计思想

结合 Montgomery 算法和 NEON 向量化的迭代 NTT 是我实现的最高效版本。NTT_Iteration_SIMD 函数在迭代 NTT 的基础上，使用 Montgomery 算法优化模运算，并对蝴蝶操作进行向量化处理。

设计了专门的旋转因子表准备函数 prepare_w_table，预先计算 Montgomery 域内的旋转因子。在蝴蝶操作阶段，我们每次处理 4 个连续的蝴蝶操作，使用 NEON 指令并行加载数据、执行 Montgomery 乘法和加减运算。边界条件处理确保模运算结果的正确性。

Montgomery 域内旋转因子预计算

```

1  static void prepare_w_table(uint32_t* W, int len, uint32_t wn, uint32_t p, const
    MontgomeryParams* params) {
2      W[0] = to_montgomery_scalar(1, params); // 1的Montgomery表示
3      for (int i = 1; i < len; i++) {
4          // Montgomery 域乘法
5          W[i] = montgomery_reduce_scalar((uint64_t)W[i-1] * wn, params);
6      }
7  }

```

NTT_Iteration_multiply_SIMD 函数整合了所有这些优化：数据转换为 Montgomery 表示，使用向量化蝴蝶操作的迭代 NTT，点乘，然后逆变换和结果转换，极大利用并行计算能力提高效率，

3.5.2 最终代码实现

使用 Montgomery 算法 neon 优化的 NTT 迭代

```

1 void NTT_Iteration_SIMD(uint32_t* a, int len, int op, uint32_t g, uint32_t p, const
2   MontgomeryParams* params) {
3   // 位逆序置换
4   for (int i = 1, j = 0; i < len-1; i++) {
5       for (int k = len >> 1; (j ^ k) < k; k >>= 1);
6       if (i < j) { uint32_t tmp = a[i]; a[i] = a[j]; a[j] = tmp; }
7   }
8   uint32_t* W = (uint32_t*)aligned_alloc(16, len * sizeof(uint32_t));
9   uint32x4_t mod_vec = vdupq_n_u32(p);
10  for (int h = 2; h <= len; h <<= 1) {
11      uint32_t root = (op == 1 ? g : qpow(g, p-2, p));
12      uint32_t root_mont = to_montgomery_scalar(root, params); // 先转域
13      uint32_t wn = montgomery_pow(root_mont, (p-1)/h, params); // 全在 Montgomery 域
14      prepare_w_table(W, h/2, wn, p, params);
15
16      for (int j = 0; j < len; j += h) {
17          int k = 0;
18          for (; k + 4 <= h/2; k += 4) {
19              uint32x4_t vA = vld1q_u32(a + j + k);
20              uint32x4_t vB = vld1q_u32(a + j + k + h/2);
21              uint32x4_t vW = vld1q_u32(W + k);
22              uint32x4_t vT = montgomery_multiply_neon(vB, vW, params);
23
24              uint32x4_t vSum = vaddq_u32(vA, vT);
25              uint32x4_t vDiff = vsubq_u32(vA, vT);
26
27              vSum = vbslq_u32(vcgeq_u32(vSum, mod_vec), vsubq_u32(vSum, mod_vec), vSum);
28              vDiff = vbslq_u32(vcgtq_u32(vT, vA), vaddq_u32(vDiff, mod_vec), vDiff);
29
30              vst1q_u32(a + j + k, vSum);
31              vst1q_u32(a + j + k + h/2, vDiff);
32          }
33          for (; k < h/2; ++k) {
34              uint32_t t = montgomery_reduce_scalar((uint64_t)a[j+k+h/2] * W[k], params);
35              uint32_t u = a[j+k];
36              uint32_t add = u + t;
37              if (add >= p) add -= p;
38              a[j+k] = add;
39              uint32_t sub = u + p - t;
40              if (sub >= p) sub -= p;
41              a[j+k+h/2] = sub;
42          }
43      }
44      free(W);
45  }
46
47  void NTT_Iteration_multiply_SIMD(int* a, int* b, int* ab, int n, int p) {
48      const int N = 2*n-1;

```

```

49  int len = 1;
50  while (len < N) len <<= 1;
51  int ni = qpow(len, p-2, p);
52  uint32_t* A = (uint32_t*)aligned_alloc(16, len * sizeof(uint32_t));
53  uint32_t* B = (uint32_t*)aligned_alloc(16, len * sizeof(uint32_t));
54  MontgomeryParams params = init_montgomery_params(p);
55
56  // 输入数据 to_montgomery_scalar
57  for (int i = 0; i < len; ++i) {
58      A[i] = (i < n) ? to_montgomery_scalar(a[i], &params) : 0;
59      B[i] = (i < n) ? to_montgomery_scalar(b[i], &params) : 0;
60  }
61  // 正向NTT
62  NTT_Iteration_SIMD(A, len, 1, 3, p, &params);
63  NTT_Iteration_SIMD(B, len, 1, 3, p, &params);
64  // 点值乘
65  for (int i = 0; i < len; ++i) {
66      A[i] = montgomery_reduce_scalar((uint64_t)A[i] * B[i], &params);
67  }
68  // 逆向NTT
69  NTT_Iteration_SIMD(A, len, -1, 3, p, &params);
70  // 输出 from_montgomery_scalar
71  for (int i = 0; i < N; ++i) {
72      ab[i] = (uint64_t)from_montgomery_scalar(A[i], &params) * ni % p;
73  }
74  free(A);
75  free(B);
76  }

```

4 ARM 下的结果与分析

<i>time/us</i>		n = 4	n = 131072		
		<i>p</i> = 7340033	<i>p</i> = 7340033	<i>p</i> = 104857601	<i>p</i> = 469762049
算法	<i>poly_multiply</i>	0.00015	95461	101199	105109
	<i>poly_multiply_neon</i>	0.00031	58407.3	64438.9	68163.6
	<i>NTT_recursion_multiply</i>	0.00542	252.097	291.666	318.798
	<i>NTT_recursion_multiply_SIMD</i>	0.01685	201.418	228.387	246.418
	<i>NTT_Iteration_multiply</i>	0.00449	83.5684	88.2875	91.4507
	<i>NTT_Iteration_multiply_SIMD</i>	0.01549	45.4782	44.4055	44.6895

表 2: ARM 下的运行结果

根据实验结果,可以对各算法的性能进行系统分析。在 $n=4$ 的小规模测试中,朴素乘法最快 (0.00015us),而各种 NTT 实现由于初始化开销较大而表现较差。可见对于极小规模问题,简单算法可能更优。

在 $n=131072$ 的大规模测试中,性能差异显著。朴素乘法需要约 95461-105109us,而 NEON 优化的朴素乘法将时间减少到 58407-68163us,提速约 39-45%。这展示了 SIMD 并行化的效果。

递归 NTT 基础版本将时间降至 252-318us,提速约 300 倍,展示了 $O(n \log n)$ 算法的巨大优势。

Montgomery 优化的递归 NTT 进一步将时间减少到 201-246us, 证明了 Montgomery 算法在减少模运算开销方面的有效性。

迭代 NTT 基础版本表现出色 (83-91 s), 比递归版本快约 3 倍, 这主要归功于消除了递归开销和更好的内存访问模式。最终的 Montgomery+NEON 优化迭代 NTT 达到最佳性能 (44-45 s), 比基础迭代版本快约 2 倍。

值得注意的是, 在 $n=131072$ 下随着模数 p 的增大 (从 7340033 到 469762049), 各算法的运行时间略有增加但幅度不大, 说明我们的实现在处理不同大小的模数时都具有良好的适应性。特别是 Montgomery 优化的版本, 对大模数的处理效率下降更小 (甚至反而更快), 可见 Montgomery 算法在处理大数模运算时具有优势。

5 移植 x86 与运行分析

5.1 移植后代码

将代码从 ARM 平台移植到 x86 平台主要涉及 NEON intrinsics 到 AVX/SSE intrinsics 的转换, 主要修改有:

1. 头文件替换: 移除了 arm_neon.h, 添加了 immintrin.h 以支持 x86 SIMD 指令。
2. SIMD 数据类型转换: 将 int32x4_t 等 NEON 类型替换为 __m128i 等 x86 SIMD 类型。
3. 指令替换: 将 vld1q_s32 等 NEON 函数替换为 _mm_loadu_si128 等 x86 等效函数。
4. 内存对齐处理: x86 对内存对齐要求更严格, 添加了 ALIGNED_ALLOC 和 ALIGNED_FREE 宏来处理不同平台的内存对齐分配。
5. Montgomery 乘法实现: 由于 x86 和 ARM 的 SIMD 指令集差异, 我们重写了向量化的 Montgomery 乘法函数, 使用 x86 的 128 位寄存器实现类似功能。

这些修改保持了算法的逻辑不变, 只是针对 x86 平台的特性进行了适配。比如蝴蝶操作的向量化实现, 虽然使用的具体指令不同, 但整体结构和优化思想保持一致。具体代码不在此处展示。

5.2 运行结果与分析

由于 cout 限制, 我只获取了 $n=4$, $p=7340033$ 的输入及答案文件, 作为正确性测试。移植后的代码在 x86 平台上通过了所有正确性测试, 说明移植正确。由于精度问题, 在 CodeBlocks 运行的结果只显示一位数, 全部运行结果都为 0us, 借助 Vtune 可以看到整个程序 (而非具体函数) 的性能: 可以

算法类型 \ 指标	用时	时钟周期/M	CPI
朴素乘法	0.036	28.8	1.200
朴素乘法 (SIMD)	0.026	21.6	0.900
递归 NTT	0.006	21.6	1.286
递归 NTT(Montgomery)	0.006	28.8	2.000
迭代 NTT	0.006	21.6	1.125
迭代 NTT(Montgomery)	0.006	26.4	1.375

表 3: Vtune 结果

看到, 所有 NTT 变体 (0.006) 均显著优于朴素乘法 (0.036) 及其 SIMD 优化版本 (0.026), 这与 ARM 平台观察到的趋势一致, 验证了 NTT 算法在理论时间复杂度上的优势。朴素乘法 SIMD 优化使执行时间从 0.036s 降至 0.026s (提升 28%), CPI 从 1.2 降至 0.9, 表明 NEON 向量化确实提高了指令级并

行度。而使用 Montgomery 的 NTT 增加 CPI 表明硬件执行效率下降，在小规模数据 ($n=4$) 下未能体现优势。这些结果与小规模测试用例的特性相关，更大规模数据可能展现不同特征。

6 实验总结和思考

本实验实现了 NTT 多项式乘法及其优化方法，从朴素乘法到高度优化的 SIMD 版本，展示了算法优化和 SIMD 加速的协同效应。可以注意到，算法级优化（从 $O(n^2)$ 到 $O(n \log n)$ ）带来的性能提升是要大于单纯的代码优化，这说明在代码设计中应首先关注算法选择。同时，SIMD 向量化、Montgomery 模约减也能带来显著的额外收益，特别是在大规模数据计算时。

通过 ARM 和 x86 双平台的实践，对跨平台向量化编程有了一定程度的理解。不同平台的 SIMD 指令集虽然概念相似，但在细节上存在诸多差异，需要仔细处理。

7 链接

Gitee 链接 [Gitee](#)